

Programmation Orientée Objet et C++

Contrôle Terminal

8 janvier 2024

Durée : 4h

Notes de cours autorisées

Accès à Moodle : <http://cours.univ-perp.fr>

Consignes importantes à lire avant de commencer :

1. Votre rendu sera composé de :
 - votre copie papier contenant vos diagrammes de classes. Cette copie sera remise à l'enseignant avant de quitter la salle.
 - d'une archive **nom-prénom.zip** contenant un dossier par exercice. Vous déposerez cette archive sur le Moodle du cours avant la fin de l'épreuve. L'absence de dépôt à la fin de l'épreuve sera considérée comme "copie blanche", vous vérifierez donc avec l'enseignant que votre travail a bien été déposé.
2. Le barème est donné à titre indicatif et est susceptible d'évoluer.
3. La qualité, la clarté et le soin apporté à la programmation seront prises en compte dans la notation.
4. Le code rendu devra pouvoir compiler. Un code ne compilant pas ne sera pas corrigé, pensez donc à vérifier que vos fichiers compilent avant de les déposer.
5. Les erreurs mémoires et les fuites mémoires seront pénalisées : **valgrind** est votre ami. On rappelle que pour utiliser **valgrind** il faut taper la commande suivante ci-dessous (pensez à compiler avec **-g**) :

```
valgrind --leak-check=full ./nom_de_votre_executable
```

L'option **--leak-check=full** n'est pas nécessaire si vous ne cherchez pas de fuites mémoires.

6. Vous organiserez votre code la façon suivante :
 - un fichier **Makefile**
 - un fichier **main.cpp** contenant le corps principal de votre programme situé au même niveau de l'arborescence que le **Makefile**.
 - un dossier **src/** contenant les fichiers sources de chaque module (**.cpp**)
 - un dossier **include/** contenant les fichiers d'en-têtes de chaque module (**.hpp**)
 - un dossier **obj/** dans lequel devront être créés les différents fichiers objets (**.o**) lors de la compilation.
 - L'absence de **Makefile** fonctionnel sera **PÉNALISÉE**.
 7. Chaque classe ou structure sera implantée dans un module différent. Vous veillerez à encapsuler au maximum les données de vos classes.
-

L'objectif de cet exercice est de concevoir et d'implanter un système de gestion informatique d'une médiathèque respectant un cahier des charges.

La médiathèque propose à ses adhérents la possibilité d'emprunter des livres, des disques et des films. On appelle dans la suite de l'énoncé *ouvrage*, tout exemplaire d'un livre, d'un film ou d'un disque de la médiathèque.

1 Ouvrages

Cahier des charges :

- les ouvrages seront identifiés par un code unique formé des trois premières lettres du titre de l'ouvrage (ou moins si le titre fait moins de 3 caractères) suivi d'un numéro¹. Ce numéro correspond à l'ordre de création dans la base de la médiathèque.
 - pour chaque livre, on mémorisera le nom de **chaque** auteur, ainsi que le titre, le nom de l'éditeur, l'année de publication ainsi que le code qui identifie l'ouvrage d'une manière unique.
 - pour chaque disque, on mémorisera le nom de chaque artiste (cela peut-être le nom d'un groupe, ou d'un chanteur, ou d'un compositeur, ou de plusieurs compositeurs, chanteurs, etc ...), le titre de l'album mais aussi le titre de chaque morceau du disque en plus du code qui identifie l'ouvrage d'une manière unique.
 - pour chaque film, on mémorisera le titre, le nom du réalisateur, l'année de sortie, la durée en heures (enregistrée comme un nombre flottant) ainsi que le code qui identifie l'ouvrage de manière unique.
 - on doit pouvoir afficher les informations concernant chaque ouvrage et obtenir le code permettant de l'identifier.
 - la base de données de la médiathèque stockera tous les ouvrages dans une même structure peu importe leur nature.
1. **(3 points)** Sur votre **copie papier**, proposez un diagramme de classes UML décrivant l'organisation des classes modélisant les ouvrages de la médiathèque en respectant le cahier des charges ci-dessus. Vous préciserez les attributs ainsi que la signature des constructeurs et méthodes.
 2. **(6 points)** Implémentez l'organisation que vous avez proposé en prenant garde à optimiser les passages de paramètres et à déclarer les méthodes pertinentes comme constantes.

Remarque : dans un premier temps vous avez seulement besoin d'implanter les méthodes répondant au cahier des charges. Vous serez libre de rajouter des méthodes à vos classes par la suite sans avoir à mettre à jour votre diagramme.

2 Prêts et adhérents

Cahier des charges

- la médiathèque stocke dans une structure tous les emprunts d'ouvrage en cours. Lorsqu'un adhérent emprunte un ouvrage, la médiathèque rajoute un emprunt dans la structure et lorsque l'ouvrage est retourné, le prêt est retiré de la structure.
- pour chaque prêt on mémorisera l'ouvrage emprunté, l'adhérent l'ayant emprunté, la date d'emprunt ainsi que la date de retour. La date d'emprunt sera stockée comme un entier entre 1 et 365 (par simplicité, on ignorera les années bissextiles), ainsi le 1er février sera le 32ème jour de l'année. La date de retour sera fixée à 3 semaines, soit 21 jours, après la date d'emprunt.
- chaque emprunt pourra être prolongé une seule fois. Dans ce cas, la date de retour sera rallongée d'une semaine (soit 7 jours). Par simplicité on laissera la date de retour dépasser 365 si nécessaire.
- pour chaque adhérent, on mémorisera le nom, le prénom, le nombre de prêts en cours ainsi qu'un historique de tous ses emprunts. Les adhérents ne peuvent pas emprunter plus de 4 ouvrages en même temps.
- l'historique des emprunts d'un adhérent doit pouvoir être mis à jour et affiché (pour chaque prêt : titre de l'ouvrage, date d'emprunt et de retour).

¹on pourra utiliser la fonction `to_string()` pour convertir un nombre en chaîne de caractères.

- les listes des adhérents et des ouvrages seront stockées au niveau de la médiathèque, on évitera donc de dupliquer inutilement leurs informations.
3. (2 points) Sur votre **copie papier**, proposez un diagramme de classes UML décrivant l'organisation des classes modélisant les prêts et les adhérents. Vous préciserez les attributs ainsi que la signature des constructeurs et méthodes.
 4. (7 points) Implémentez l'organisation que vous avez proposé en prenant garde à optimiser les passages de paramètres et à déclarer les méthodes pertinentes comme constantes.

Remarque : dans un premier temps vous avez seulement besoin d'implanter les méthodes répondant au cahier des charges. Vous serez libre de rajouter des méthodes à vos classes par la suite sans avoir à mettre à jour votre diagramme.

3 Médiathèque

Cahier des charges

- la médiathèque stockera tous les ouvrages dans une même structure indépendamment de leur nature. On accèdera à chaque ouvrage par son code d'identification. On pourra rajouter des ouvrages dans la structure mais pas en retirer.
 - la médiathèque gardera en mémoire une liste de tous ses adhérents. De nouveaux adhérents doivent pouvoir être rajoutés à la liste.
 - les prêts, ceux en cours et ceux passés, sont enregistrés dans l'historique des emprunts des adhérents. La médiathèque garde également en mémoire une liste de tous les emprunts en cours où chaque emprunt est identifié par le code de l'ouvrage emprunté.
 - Les prêts en cours seront donc stockés à la fois dans la liste de la médiathèque et dans l'historique de l'adhérent. En revanche, une fois l'ouvrage retourné, le prêt sera retiré de la liste des emprunts en cours.
 - chaque ouvrage est possédé en un unique exemplaire, ainsi on ne pourra pas emprunter un ouvrage si celui-ci est déjà emprunté.
 - on doit pouvoir afficher la liste des ouvrages de la bibliothèque ainsi que les informations d'un adhérent, identifié par son nom (nom, prénom, historique des emprunts).
5. (2 points) Une partie de la classe médiathèque a déjà été imaginée ci-dessous.

mediatheque
- liste_ouvrages : std::map<std::string, ...> - liste_adherents : ... - liste_emprunts : std::map<..., ...>
- ...est_emprunte(...) + ~mediatheque() + ...ajout_adherent(...) + ...ajout_ouvrage(...) + ...emprunt(...) + ...retour(...) + ...affiche_ouvrages(...) + ...affiche_info_adherent(...)

Sur votre **copie papier**, complétez le diagramme UML ci-dessus en remplissant les points de suspension de sorte à respecter le cahier des charges.

6. (8 points) Implémentez la classe `mediatheque` décrite ci-dessus.
7. (2 points) Rajoutez une méthode `void ajout_ouvrages(string file_name)` à la classe médiathèque permettant de rajouter des ouvrages à la liste de la médiathèque à partir d'un fichier dont le nom sera passé en paramètre. Le fichier aura le format suivant :

```
# Livre
titre
auteur1 ; auteur2 ; auteur3 ; ...
editeur
annee de publication
# Disque
titre
artiste1 ; artiste2 ; artiste3 ; ...
morceau1 ; morceau2 ; morceau3 ; ...
# Film
titre
realisateur
annee de sortie
duree
# Disque
...
```

Le fichier `ouvrages.dat` fourni sur le Moodle est un exemple d'un tel fichier.